

MineCRust Project Report

Progress Description & Overview

My main goal for this project was building a voxel game engine from scratch using Rust and modern OpenGL. I wanted to recreate the core features that make games like Minecraft work. I focused on making terrain generate on its own, keeping the game running smoothly, and building a system that could be easily expanded later. I wanted the game to create endless terrain, have lighting that changes naturally with the environment, and let users interact with everything in real-time without the game slowing down.

I'm happy with how things turned out. The chunk-based rendering system creates endless terrain using Perlin noise algorithms. The engine now has a working day-night cycle that changes both how the sky looks and how bright the terrain is. Players can fly around the world with a camera, add and remove blocks, and control things through a custom command prompt interface.

The biggest problems I faced were with mesh generation and keeping track of coordinates. Face-culling was harder than I expected because I need to check blocks next to each other across chunk borders to avoid drawing hidden surfaces. Getting this working correctly took a lot of debugging time. The texture atlas also needed careful work, since calculating UV coordinates for different block types and sides required exact precision to prevent visual glitches.

Connection to Our Class

This project shows several important computer graphics ideas we learned about in class. I manually control the rendering pipeline by building meshes for each chunk, setting up vertex attributes like position, texture coordinates, and normals, then sending them to the GPU using VBOs and VAOs.

Visibility calculations happen on the CPU side to make things faster. The mesh builder looks at nearby blocks before adding faces to the mesh, making sure that hidden faces never get sent to the rendering pipeline. This is really important for voxel engines where most block faces are usually hidden from view.

The lighting system uses custom shaders based on concepts we studied. The fragment shader calculates the dot product between surface normals and the sun's direction to figure out how bright the diffuse lighting should be, using the Lambertian reflectance model. An ambient lighting component changes with the time of day, showing both global and local lighting principles. Texture sampling works through a texture atlas where I calculate specific UV coordinates for each block face to grab the right part of the combined texture. Matrix transformations are used a lot for camera view and projection matrices, and also for rotating block vertices when building meshes to support placing blocks in different directions.

Key Features & Images

The final system has several advanced features working together. The endless procedural terrain keeps generating as the player moves around, with different biomes including forests, deserts, and plains. These biomes are chosen based on temperature and humidity maps that come from noise functions.

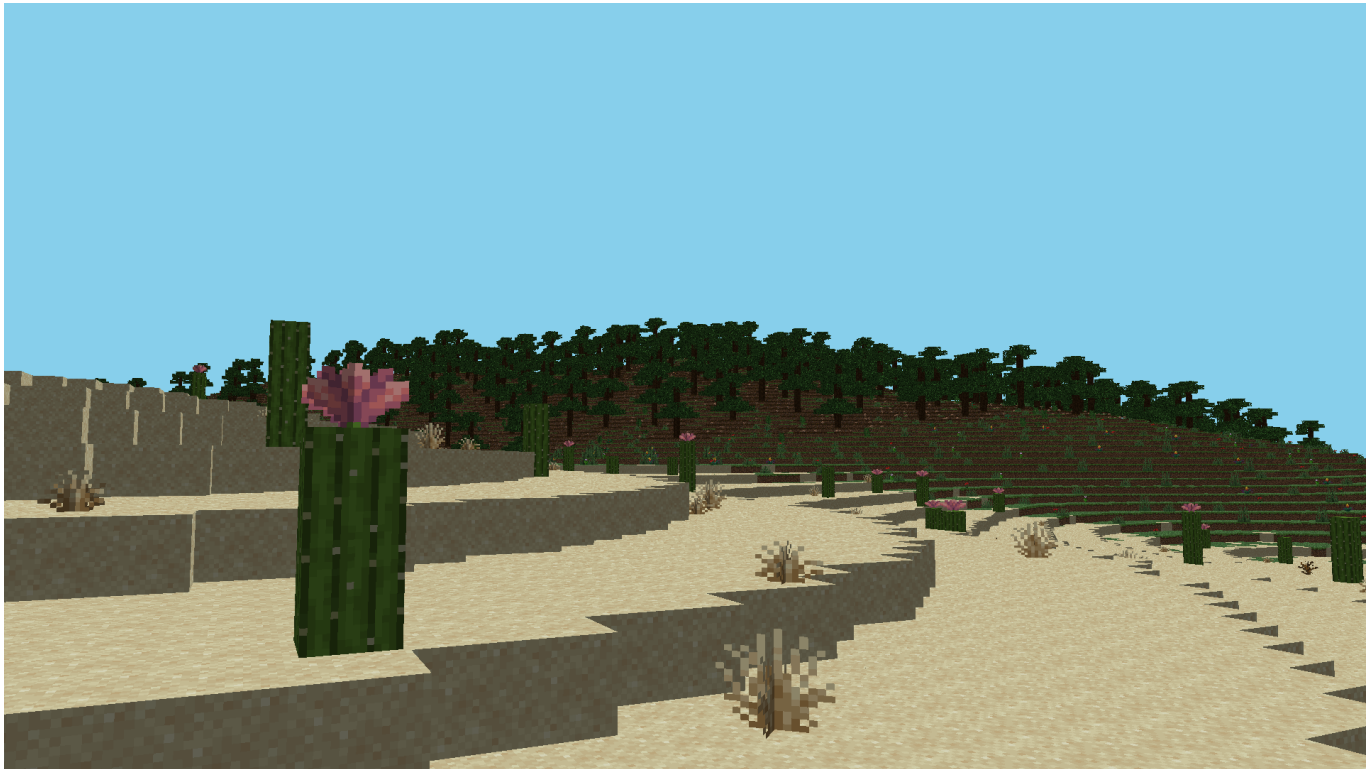


Figure 1: Endless terrain generation showing different biomes and plant life.

The dynamic day-night cycle makes the sun and moon move across the sky in real-time. Sky colors shift smoothly from dawn through noon to dusk and night, while the brightness of the world changes to match. This creates a feeling of time passing and adds depth to the atmosphere.



Figure 2: The lighting system responding to the time of day.

The building system lets players change the terrain by breaking or placing blocks. The system supports placing blocks with specific rotations, like logs pointing in different directions. This works by changing vertices during mesh construction. An in-game command interface lets users change game settings like time of day, switch game modes, or pick different block types without restarting the game.



Figure 3: Block placement and modification features.

Resources Used

Rust was the foundation for the entire project. Several libraries helped with low-level operations. The `gl` crate gave me raw OpenGL bindings, while `glfw` handled window creation, context management, and input handling. Math operations with vectors and matrices were done using the `cgm` library. Terrain generation used the `noise` crate for Perlin noise functions. To keep performance good, the `tokio` async runtime moved chunk generation to background threads, stopping the main thread from getting blocked. The 'image' library handled loading texture files.

I also looked at the decompiled Java code of Minecraft version 1.21 and Luanti (formerly Minetest), an open-source C++ version. These references helped me understand chunk logic, block placement systems, terrain generation, and decoration generation including trees, grass, and flowers. Looking at these existing games gave me insights that helped with several design choices.

Future Work

Several areas could be improved or expanded. Better lighting features, especially shadow mapping and point light sources like torches, would make the visuals much more realistic and add depth. The current lighting system works but could be a lot better with these additions.

Adding physics would be another good next step. Setting up collision detection and player physics including gravity and jumping would turn the engine from a technical demo into an actual playable game. These systems are essential for player interaction and moving around the world.

Water rendering needs work. Right now the water blocks are static and look flat. Adding fluid dynamics or at least animated flowing effects would make it look much more realistic. Also, adding more biome types would make the terrain more interesting. Underground cave systems would add vertical variety and give players more reasons to explore beyond just the surface.

Making things run faster is always important. While face-culling helps a lot, frustum culling and level-of-detail systems for faraway chunks could make frame rates even better. Trying out compute shaders for mesh generation instead of CPU-based construction might give better performance. In the long term, multiplayer support would be an ambitious but exciting addition, though the networking complexity would probably be big enough to count as its own separate project.